

IF IN TEST CLASS — CHEAT SHEET

Two totally different things called “if in the test class”

1. Assumptions.assertTrue() — SKIPS the test

```
import static org.junit.jupiter.api.Assertions.*;

@Test
void test() {
    assertTrue(condition); // false -> test SKIPPED, not failed
    assertEquals(...);
}
```

- `assertTrue(x)` → skip if x is false
- `assertFalse(x)` → skip if x is true
- Skipped ≠ passed ≠ failed — separate result category.

2. Plain Java if inside a @Test method — normal branching

```
@Test
void test() {
    if (cond) {
        assertEquals(a, result);
    } else {
        assertEquals(b, result);
    }
}
```

- Bad practice (one test, two scenarios) but valid Java.
- Treat exactly like an `if` in production code for CFG / complexity questions.
- “Better” fix if asked: split into 2 separate `@Test` methods, one per branch.

Counting it for Cyclomatic Complexity

$V(g) = \text{decisionpoints} + 1$

Every `if`, `else if`, `for`, `while`, `case`, `&&`, `||` = +1 decision point. Doesn't matter if it's in the test class or the class under test — count it the same way.

Branch coverage rule

One `if/else` = need ≥ 2 test cases (one true path, one false path). Nested / multiple conditions → one test per path.

Exam gut-check

- “Draw CFG of this test method” → draw the `if` like normal code: one node, two edges.
- “Why does this test skip?” → look for `assertTrue/assertFalse`, not `assert`.
- “Fix this test” (has `if/else` with asserts) → split into 2 tests, name each after the branch.